FORMATO DE DESARROLLO E INTEGRACIONES

ORIGINACION CROSSCORE

STF GROUP S.A.

INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, viernes, 16 de febrero de 2024

TABLA DE CONTENIDO

Cali, martes, 14 de noviembre de 2023 1
INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Descripción del Requerimiento 4
1.2 Detallar componentes existentes 5
1.3 Detallar funcionalidades 6

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

1. DETALLE TECNICO

1.1 Descripción del Requerimiento

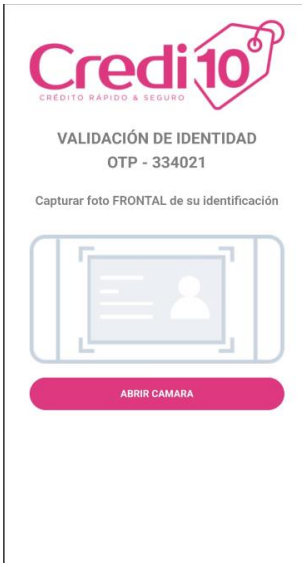
1.1.1 Requerimiento

Realizar todo el proceso de inscripción del cliente y se realiza una serie de validaciones de seguridad para asignar un cupo de crédito por un valor en pesos colombianos.

1. Capturar los datos básicos del cliente
2. Realizar proceso de Aceptación de tratamiento de datos con la captura de fotos al documento de identidad y una selfie.
3. Realizar validaciones de Inspektor, Datacredito, Fecha de Expedición de documento, Correo Electrónico, Celular
4. Permitir la aceptación de cupo

Ver el requerimiento en el link: RF-ADM- 001 - FT-SL-001-Módulo_Originación.docx

1.1.2 Flujo del Proceso

<table>
<tr><td>1
 
</td><td>2
 
</td></tr>
<tr><td>3</td><td>4</td></tr>
</table>



1.2 Detallar componentes existentes

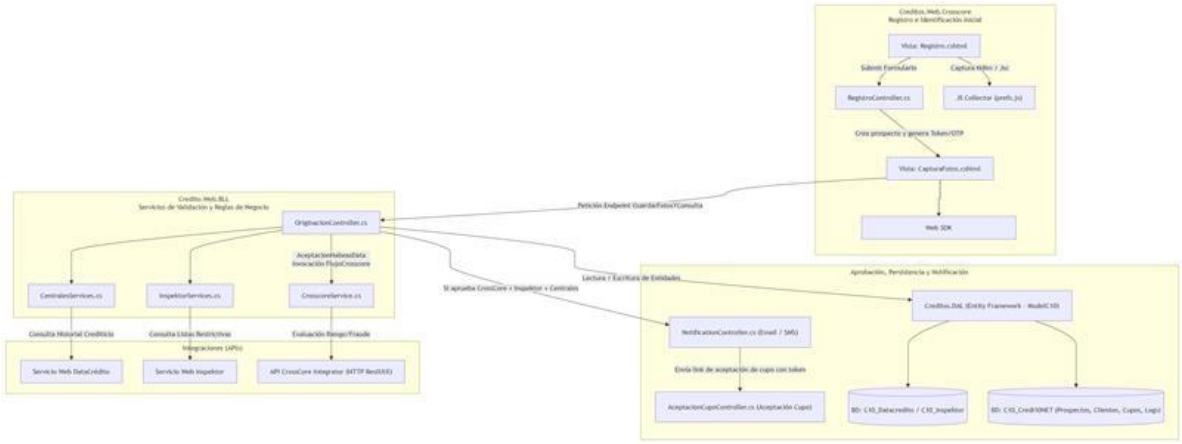
SOLUCION	
Creditos.Web.Crosscore • Controladores ○ RegistroController.cs ○ AceptacionCupoController.cs • Vistas ○ Registro.cshtml ○ CapturaFotos.cshtml ○ ValidarTokenAceptacion.cshtml ○ HabeasData.cshtml	Clases y vistas implementadas para cumplir con el propósito registrar un prospecto, validar mediante el servicio de Crosscore, crear y aceptar el cupo.
Creditos.BLL • Prospectos.cs • CuposController.cs • OriginacionController.cs • ServiciosController.cs • InspektorServices.cs • CentralesServices.cs • NotificationController.cs • SecurityUtil.cs • RestUtil.cs	Clases implementadas para realizar las consultas a DataCredito, Inspecktor, validaciones de identidad como: celular, correo, fecha expedición, numero de celular, correo. Clases para la notificación al cliente.
Creditos.DAL • ModelC10 ○ C10Model.edmx	Modelo de datos de entidad.

Creditos.DTO • CentralesRequest • CentralesLoginRequest • InspektorLoginRequest • InspektorRequest • SolicitudFormularioDTO • AceptacionReferenciasDTO • HabeasDataDTO • PagareDTO	Clases de transferencia de datos
StyleLoader.css	Estilos pantalla carga
StylesRegistro.css	Estilos vista registro
AutoNumeric	Librería Formato de números
Notification.js	Contiene los métodos que se utilizan para mostrar las alertas a los usuarios.
prefs.js	Proporcionado por Experian. métodos de JavaScript para la extracción de la información del dispositivo.
BASE DE DATOS	
C10_Credi10NET.dbo.tbl_calificacion	Tablas
C10_Credi10NET.dbo.tbl_cliente_correos	Tablas
C10_Credi10NET.dbo.tbl_cliente_direcciones	Tablas
C10_Credi10NET.dbo.tbl_cliente_fotos	Tablas
C10_Credi10NET.dbo.tbl_cliente_telefonos	Tablas
C10_Credi10NET.dbo.tbl_clientes	Tablas
C10_Credi10NET.dbo.tbl_cupos	Tablas
C10_Credi10NET.dbo.tbl_historico_estados_cupos	Tablas
C10_Credi10NET.dbo.tbl_historico_monto_cupos	Tablas
C10_Credi10NET.dbo.tbl_otp_aceptacion	Tablas
C10_Credi10NET.dbo.tbl_otp_prospectos	Tablas
C10_Credi10NET.dbo.tbl_parametros	Tablas
C10_Credi10NET.dbo.tbl_parametros_mensajes	Tablas
C10_Credi10NET.dbo.tbl_prospectos	Tablas
C10_Credi10NET.dbo.tbl_rutas_documentos	Tablas
C10_Credi10NET.dbo.tbl_tipos_documentos	Tablas

C10_Datacredito.dbo.tbl_consulta_centrales	Tablas
C10_Inspektor.dbo.tbl_consulta_inspektor	Tablas

1.3 Detallar funcionalidades

1.3.1 Diagrama de Componentes



1.3.2 MER

1.3.3 Parametrizaciones

Parametrizaciones desde base de datos.

- Tbl_paramepunttros, Campo Key, Valor EncendidoInspektor: Sirve para habilitar o deshabilitar la consulta del prospecto en inspecktor, si está en True se consulta, si está en False se omite la consulta.

- Tbl_parametros, Campo Key, Valor EncendidoSMS: Sirve para encender o apagar el envió de sms con el otp de aceptación del cupo Asi: ”Para continuar con la solicitud su otp es el siguiente {0}.” si esta en True, se notifica por sms, si está en false no.

- Tbl_parametros, Campo Key, Valor EncendidoValidacionCorreo: Sirve para encender o apagar la validación del correo electrónico en el proceso de originacion, si está en true se valida el correo electrónico, si está en false se omite la validacion

- Los mensajes que se muestran en las vistas durante el proceso de originacion están parametrizados en la tabla *[tbl_parametros_mensajes]* y tiene configurado en la columna '*grupo'* la palabra 'Originación', los mensajes que se muestran en este proceso a continuación:

id_parametro	descripcion	grupo	key_	valor	codigo
1	Ya acepto Habeas (HabeasDataAceptado)	Originación	HabeasAceptado	Ya acepto el HabeasData, el estado del prospecto...	1001
2	No acepto Habeas data (HabeasDataAceptado)	Originación	HabeasNoAceptado	No acepto el HabeasData, el estado del prospecto...	1002
3	Otp validado con exito, respuesta al correo (Val...	Originación	RespuestaAlCorreo	La respuesta a la solicitud de cupo, fue enviada a t...	1003
4	Generando nuevo OTP (EnviarCorreoSolicitud)	Originación	NuevoOtpProspecto	Se creó un nuevo código por favor revise su corre...	1004
5	Supero limite de intentos (ValidarOtpsCreados)	Originación	LimiteMaximoIntentos	Se ha superado el límite máximo de intentos, el pro...	1005
6	Solicitud aprobada (AceptacionHabeasData)	Originación	InformaSolicitudAprobada	La respuesta a la solicitud de cupo, fue enviada a t...	1006
7	Solicitud fallo (AceptacionHabeasData)	Originación	InformaFalloSolicitud	La respuesta a la solicitud de cupo, fue enviada a t...	1007
8	Solicitud rechazada (AceptacionHabeasData)	Originación	InfomaSolicitudRechazada	La respuesta a la solicitud de cupo, fue enviada a t...	1008
9	Otp Incorrecto intentos (ValidarOtpFlujoCajera)	Originación	OtpIncorrectoIntentos	El OTP es incorrecto, tiene {0} intentos mas	1009
10	Otp Incorrecto (ValidarOtpFlujoCajera)	Originación	OtpIncorrecto	El OTP es incorrecto	1010
11	TokenNoExiste	Originación	TokenNoExiste	El token no existe por favor comunicarse con nues...	1011
12	Aviso referencias diferentes (ValidarOtpAcepta...	Originación	ReferenciasDiferentes	La referencia personal y familiar debe ser una pers...	1012
13	Aviso celular referencias diferentes (ValidarOtp...	Originación	CelularReferenciasDifere...	El celular de la referencia personal y familiar deben ...	1013
14	Otp Incorrecto aceptacion cupo (ValidarOtpAc...	Originación	OtpAceptacionIncorrecto	El otp no coincide	1014
15	Aviso ya puede comprar (OtpAceptacionCoinci...	Originación	EmpezarCompras	Ya puedes empezar a comprar con tu cupo en las t...	1015
59	Excepcion del metodo CrearCupo	Originación	ErrorCreandoCupo	Error al crear cupo	2044
61	Advertencia ya aceptó el cupo	Originación	YaAceptoElCupoEstado	Ya acepto el Cupo, el estado del cupo es:	2046
62	Advertencia NO aceptó el cupo	Originación	NoAceptoElCupoEstado	No acepto el Cupo, el estado del cupo es:	2047
112	El celular de las referencias debe ser diferente ...	Originación	CelularReferenciaDiferen...	El celular de las referencias debe ser diferente al c...	1018

1.3.4 Proceso

El presente requerimiento se desarrolla bajo la solución Creditos.sln, de esta solución los proyectos que abarcan el presente requerimiento son cuatro Creditos.Web.Crosscore, Creditos.BLL, Creditos.DAL y Creditos.DTO.

Es importante mencionar que el proceso de Originacion Crosscore y el proceso de Originacion se unen en la capa de BLL. Crosscore incluye la validación por medio de fotos del documento de identidad por ambas caras, consumo y validación de la respuesta de Crosscore y el punto de conexión con el flujo Originacion se da cuando se aprueba Crosscore y se continua el proceso con el consumo del api de Inspector y Datacredito ya que ambos procesos realizan los consumos y validaciones por medio del método de BLL ConsultaInspectorCentrales que se encarga de consultar a Inspector y Datacredito y en caso de que se apruebe y pase las validaciones de crear el cliente, el cupo.

El proceso inicia con la inscripción de prospectos en la vista Registro.*cshtml* del controlador RegistroController.cs, la vista contiene el formulario de registro. Al formulario de registro se añaden dos campos adicionales solicitados en el api de Crosscore para la validación del dispositivo, los campos son Hdim Jsc, se añaden a partir de la implementación del JavaScript Collector proporcionado por Experian.

La implementación JavaScript Collector, se describe en el manual proporcionado por el tercero que se presenta en el siguiente link, consiste en tomar el archivo proporcionado "deviceinsight-collector-javascript-8.0.0-fraud-extended" en base a este archivo se creó uno nuevo con el nombre prefs.js, y se incluyó en la vista en donde se encuentra el formulario de registro, en este caso Create.cshtml con el siguiente código:

```
<script type="text/javascript">
    //IMPLEMENTACION JSCOLLECTOR
    (function (x, p, e, r, i, a, n) {
        x["eco"] = i;
        x[i] = x[i] || function () { (x[i].c = x[i].c || []).push(arguments) };
        a = p.createElement(e); a.src = r; a.async = 1; n = p.getElementsByTagName(e)[0];
        n.parentNode.insertBefore(a, n);
    })(window, document, "script", "../Scripts/prefs.js", "xp");
    xp({
        formSelector: "Form",
        validateOnSubmit: true,
    });
```

Con lo anterior, en el formulario con Id #Form que contiene todos los campos del prospecto se añaden los campos requeridos por Crosscore Hdim y Jsc para la validación del dispositivo.

Las validaciones de datos únicos del formulario para que no se repitan: número de identificación, celular, correo electrónico en *[tbl_prospectos]* y *[tbl_clientes]* se realizan en JavaScript mediante el consumo de endpoints con Ajax. Los métodos que se consumen son de *RegistroController* y corresponden a *BuscarProspecto(), BuscarCelular(), BuscarCorreo()*, en caso de que los datos ingresados en el formulario ya estén registrados se muestra una alerta.

Una vez diligenciado el formulario, se da clic sobre el botón de Culminar y con esta acción se consume el endpoint “Create”, se crea el objeto *SolicitudFormularioDTO* y se inserta en *[tbl_prospectos]* la información suministrada, se crea un token alfanumérico y se crea un OTP en la tabla *[tbl_otp_prospectos].* Se envía el link que apunta a la vista de la captura de las fotos con el token al correo electrónico junto con una plantilla HTML que esta alojada en BD en la tabla *[tbl_configuracion_mensajeria*] bajo el *key_ “EmailMessageSolicitud*” él envió del correo se realiza haciendo uso del método *EnviarCorreoSolicitudCrosscore()* del controlador *NotificationController.cs*

El proceso continua con la captura de las fotos del documento de identidad y la selfi en la vista CapturaFotos.cshtml en ella se realiza la implementación y el uso del “JavaScript Web SDK v11.5.1” dado por Expirian, código que proporciona los métodos necesarios para integrar la funcionalidad de captura y procesamiento de imágenes. Los archivos de JavaScript que se tuvieron que incluir en la vista CapturaFotos.cshtml son los que se mencionan a continuación: “AcuantJavascriptWebSdk.min.js”, “AcuantCamera.min.js”, “AcuantPassiveLiveness.min.js”, archivos que se describen a detalle en la documentación proporcionada por el tercero: Especificacion-JavaScritp-ValidacionDocumental, de esta implementación es importante resaltar que se configuro las credenciales que fueron proporcionadas y son necesarias para la validación de “Vida Pasiva”(define si en la foto hay una persona) y también para la validación del “FaceMatch”

El proceso de captura de las fotos inicia en la vista CapturaFotos.cshtml esta vista contiene:

- Input oculto del token del prospecto con id #token.
- Input oculto del número de cedula del prospecto #cedulaProspecto.
- Los inputs ocultos para almacenar temporalmente las fotos capturadas: #inputFotoFrontal, #inputFotoTrasera, #inputSelfie.

- Párrafo etiqueta con id #leyenda, se usa para mostrar al usuario la indicación del proceso de captura.
- Imagen etiqueta <img> con id #indicacion, se usa para mostrar visualmente la indicación del paso que debe realizar el usuario.
- Variables de control para el flujo de la captura: “fotoFrontalTomada”, “fotoTraseraTomada”, “selfieTomada” y “FaceMatchAprobado”
- Botón “ABRIR CAMARA” con id #CapturaYa

```
@model Creditos.DTO.Originacion.SolicitudFormularioDTO
<body>
    @if (result.Status)
    {

        <div class="container contenedor">
            @Html.AntiForgeryToken()
            <input value="@ViewBag.tokenProspecto" name="token" id="token" hidden />
            <input value="@ViewBag.cedulaProspecto" name="cedulaProspecto" id="cedulaProspecto" hidden />

            <input type="hidden" id="inputFotoFrontal" name="fotoFrontal" />
            <!-- Input oculto para la foto trasera -->
            <input type="hidden" id="inputFotoTrasera" name="fotoTrasera" />
            <!-- Input oculto para la selfie -->
            <input type="hidden" id="inputSelfie" name="selfie" />
            <img src="~/images/assets/logo_fucsia.png" alt="Logo empresarial" class="img-fluid logo" />
            <br />
            <p id="" class="msj-validacion"><strong>VALIDACIÓN DE IDENTIDAD</strong> </p>
            <p id="" class="msj-validacion"><strong>OTP - @result.Data </strong> </p>
            <br />
            <p id="leyenda" class="leyenda-indicacion"></p>
            <img src="~/images/assets/IDfront2.png" alt="Indicación" class="img-fluid indicacion" id="indicacion" />
            <button id="CapturaYa" class="btn credi10-captura-btn">ABRIR CAMARA</button>
```

Los mensajes que se muestran durante el proceso de captura son los que están configurados en la tabla [tbl_parametros_mensajes] y tienen en la columna grupo el valor ‘CrosscoreFlujoCaptura’, estos mensajes son enviados a la vista por medio de ViewBag y se almacenan en variables de JavaScript para su posterior uso en el flujo de la captura.

key_	Valor	Codigo
leyendaFrente	Capturar foto FRONTAL de su identificación	130005
leyendaFrenteError	Capturar foto FRONTAL de la identificación. Error al capturar la foto. Asegúrese que tenga buena iluminación, sin sombras. Intente nuevamente	130006
leyendaPosterior	Capturar foto POSTERIOR de su identificación	130007
leyendaPosteriorError	Capturar foto POSTERIOR de su identificación. Error al capturar la foto. Asegúrese que tenga buena iluminación, sin	130008

	sombras. Intente nuevamente	
leyendaSelfie	Capturar foto tipo SELFIE	130009
leyendaSelfieError	Capturar foto tipo SELFIE. Error al capturar la foto. Asegúrese que tenga buena iluminación, sin sombras. Intente nuevamente	130010

La función de JavaScript FlujoCaptura() es la encargada de gestionar la captura de las 3 fotos y gestionar los elementos que se muestran en la vista al usuario.

Cuando se da clic en el botón #CapturaYa, se invoca al método startCamara() que internamente se encarga de iniciar la cámara con el método incluido en el SDK AcuantCamera.startManualCapture() este sirve para capturar las fotos del documento de identidad, si se va a capturar la foto tipo selfi se invoca el método openFaceCaptureCamera.

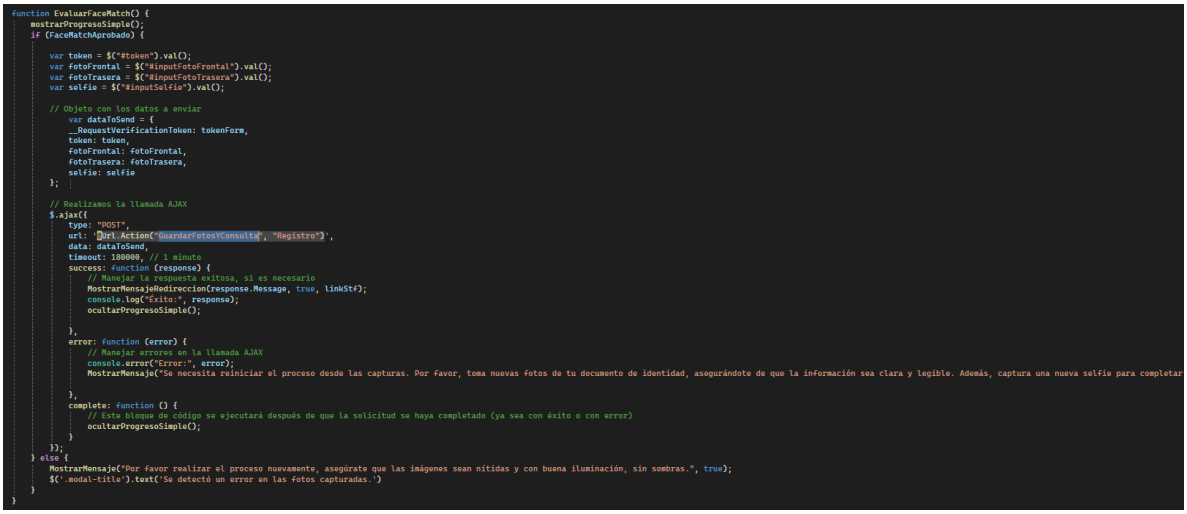
La cámara se abre y permite al usuario capturar la foto, una vez se toma la foto se ejecuta la función de CallBack: OnCroped, que recibe el objeto con la foto recortada y la información de la foto capturada, aquí dentro se invoca el método “FlujoCaptura” que también recibe el objeto que se envió a OnCroped.

“FlujoCaptura” internamente evalúa las variables de control, para identificar cual es la foto que se tomó (fotoFrontalTomada, fotoTraseraTomada, selfieTomada), si son fotos del documento entonces se pasara a evaluar la respuesta con el método “CumplirMetricas” que evalúa los parámetros de nitidez y deslumbramiento de la foto, donde la nitidez o “sharpness” debe superar el valor 50, y el deslumbramiento “glare” debe ser mayor o igual a 50, ya que un valor de 100 indica que la foto no tiene deslumbramiento y 0 que si tiene deslumbramiento. De acuerdo con el resultado de la evaluación de las métricas, se cambia el texto y la imagen de las indicaciones mostradas al usuario y se establece fotoFrontalTomada, fotoTraseraTomada en True según la captura que está realizando, si es el proceso de captura de la foto tipo selfi, entonces se captura la foto y se envía a “AcuantPassiveLiveness.getLiveness” que valida que la foto que se envía corresponda a una persona, se verifica que esté retornando en la respuesta “Live” en result.LivenessResult.LivenessAssessment; Cuando las métricas se cumplen o la validación de la persona en la selfi se cumple se asigna a cada input oculto la foto que corresponde.

Caba anotar que en caso de que las validaciones de las métricas no se cumplan o el resultado de “getLiveness” sea diferente de “Live”, entonces FlujoCaptura se encargara de mostrar el mensaje de error que corresponda segun la foto que esta tomando, para que el usuario vuelva a realizar la captura.

Una vez se capturan todas las fotos se llama al método de Javascript CargarImagenesFaceMatch() que recibe la foto frontal del documento y la foto tipo selfi para validar que en las fotos corresponda la misma persona, esto se realiza con ayuda del SDK

proporcionado usando el método “getFaceMatch”, este mismo método se encarga de recibir una función callback “onFaceMatch” que establece el valor de la variable “FaceMatchAprobado” en true o false según el resultado de la validación de la coincidencia en las fotos, luego se invoca el método “EvaluarFaceMatch()”, que verifica la variable “FaceMatchAprobado”: si la variable “FaceMatchAprobado” es false entonces se redirige al usuario al inicio recargando la página para que reinicie el proceso de captura de las fotos ya que no se encontro la coincidencia en el rostro, si la variable “FaceMatchAprobado” es true se pasa a consumir el endpoint “GuardarFotosYConsulta” de RegistroController.cs, recibe el token del prospecto, la foto frontal y foto posterior del documento en Base64 y la selfi en Base64, “GuardarFotosYConsulta” registra las fotos del prospecto, consume Crosscore y en caso de que Crosscore sea aprobado realiza el consumo de las Apis de Inspector y Datacredito.



```
function EvaluarFaceMatch() {
    mostrarProgresoSimple();
    if (FaceMatchAprobado) {

        var token = $("#token").val();
        var fotoFrontal = $("#inputFotoFrontal").val();
        var fotoTrasera = $("#inputFotoTrasera").val();
        var selfie = $("#inputSelfie").val();

        // Objeto con los datos a enviar
            var dataToSend = {
            __RequestVerificationToken: tokenForm,
            token: token,
            fotoFrontal: fotoFrontal,
            fotoTrasera: fotoTrasera,
            selfie: selfie
        };

        // Realizamos la llamada AJAX
        $.ajax({
            type: "POST",
            url: '@Url.Action("GuardarFotosYConsulta", "Registro")',
            data: dataToSend,
            timeout: 180000, // 1 minuto
            success: function (response) {
                // Manejar la respuesta exitosa, si es necesario
                MostrarMensajeRedireccion(response.Message, true, linkStf);
                console.log("Éxito:", response);
                ocultarProgresoSimple();

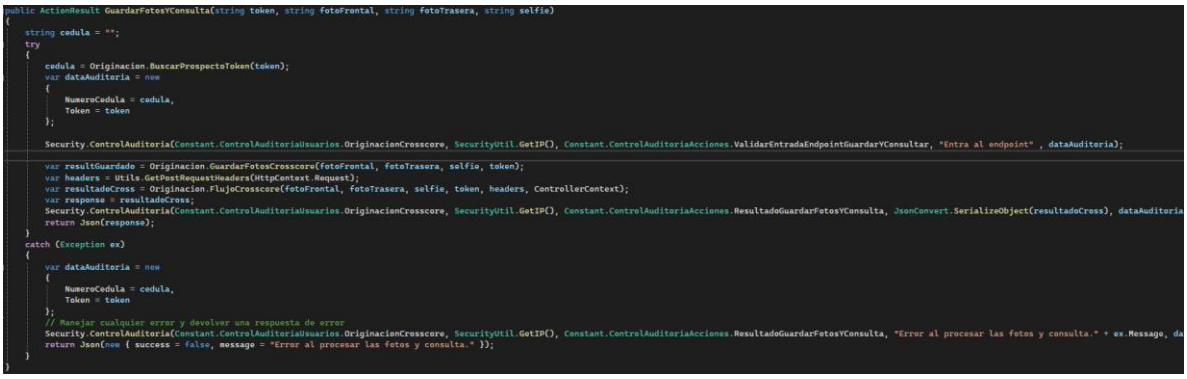
            },
            error: function (error) {
                // Manejar errores en la llamada AJAX
                console.error("Error:", error);
                MostrarMensaje("Se necesita reiniciar el proceso desde las capturas. Por favor, toma nuevas fotos de tu documento de identidad, asegurándote de que la información sea clara y legible. Además, captura una nueva selfie para completar

            },
            complete: function () {
                // Este bloque de código se ejecutará después de que la solicitud se haya completado (ya sea con éxito o con error)
                ocultarProgresoSimple();
            }
        });
    } else {
        MostrarMensaje("Por favor realizar el proceso nuevamente, asegúrate que las imágenes sean nitidas y con buena iluminación, sin sombras.", true);
        $('.modal-title').text('Se detectó un error en las fotos capturadas.')
    }
}
```

“GuardarFotosYConsulta” invoca al método de BLL Originacion.GuardarFotosCrosscore() que guarda las fotos en el disco del servidor y almacena la ruta de cada foto en la tabla [tbl_validacion_identidad]. La ruta se define partir de lo parametrizado en la *[tbl_parametros],* se toma la columna *value*, de los *key_ RutaArchivosClientes* y *CarpetaActualArchivosClientesSemestre* a los cuales se les concatena la cedula de la cliente encriptada con sha256(), por lo que la ubicación de los archivos está dada por:

RutaArchivosClientes + CarpetaActualArchivosClientesSemestre + "\\" + Sha256(cedulaCliente) + “\\ValidacionIdentidad”

Luego de guardar las fotos se llama al método Utils.GetPostRequestHeaders para recuperar un diccionario con todos los headers de la solicitud ya que son un parámetro para el consumo del api de Crosscore, por último, se llama al método Originacion.FlujoCrosscore que recibe la foto frontal, la foto trasera, la selfi, el token del prospecto, el diccionario con los headers, y el contexto del controlador de registro, este último para poder crear el PDF de la aceptación con base a HabeasDataPersonalizado.cshtml.



```
public ActionResult GuardarFotosYConsulta(string token, string fotoFrontal, string fotoTrasera, string selfie)
{
    string cedula = "";
    try
    {
        cedula = Originacion.BuscarProspectoToken(token);
        var dataAuditoria = new
        {
            NumeroCedula = cedula,
            Token = token
        };

        Security.ControlAuditoria(Constant.ControlAuditoriaUsuarios.OriginacionCrosscore, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.ValidarEntradaEndpointGuardarYConsultar, "Entra al endpoint" , dataAuditoria);

        var resultGuardado = Originacion.GuardarFotosCrosscore(fotoFrontal, fotoTrasera, selfie, token);
        var headers = Utils.GetPostRequestHeaders(HttpContext.Request);
        var resultadoCross = Originacion.FlujoCrosscore(fotoFrontal, fotoTrasera, selfie, token, headers, ControllerContext);
        var response = resultadoCross;
        Security.ControlAuditoria(Constant.ControlAuditoriaUsuarios.OriginacionCrosscore, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.ResultadoGuardarFotosYConsulta, JsonConvert.SerializeObject(resultadoCross), dataAuditoria
        return Json(response);
    }
    catch (Exception ex)
    {
        var dataAuditoria = new
        {
            NumeroCedula = cedula,
            Token = token
        };
        // Manejar cualquier error y devolver una respuesta de error
        Security.ControlAuditoria(Constant.ControlAuditoriaUsuarios.OriginacionCrosscore, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.ResultadoGuardarFotosYConsulta, "Error al procesar las fotos y consulta." + ex.Message, da
        return Json(new { success = false, message = "Error al procesar las fotos y consulta." });
    }
}
```

“FlujoCrosscore” de OriginacionController.cs de la capa BLL: es el método central de este requerimiento ya que se encarga de realizar la petición al servicio de Crosscore y luego de ello dependiendo del resultado, pasar a consumir las apis de Inspector y Datacredito para posteriormente crear el cupo y el cliente.

```
public OperationResponse FlujoCrosscore(string frontalB64, string posteriorB64, string selfieB64, string token, Dictionary<string, string> headers, ControllerContext context)
{
    var response = new OperationResponse();
    try
    {
        var prospectoEstudio = Connection.ContextC10.tbl_prospectos.FirstOrDefault(p => p.token == token);
        ActualizarEstadoProspecto(prospectoEstudio, Constant.MotivosProspecto.Crosscore, (int)EstadosProspecto.En_Proceso);
        var prospecto = Connection.ContextC10.tbl_prospectos.FirstOrDefault(p => p.token == token);
        var numeroIdentificacion = prospecto.numero_identificacion;
        _ = CrearDirectorioCliente(prospecto);
        var otps = Connection.ContextC10.tbl_otp_prospectos.Where(o => o.fk_id_prospecto == prospecto.id_prospecto).ToList();
        var otpVigente = otps.FirstOrDefault(a => a.fk_id_prospecto == prospecto.id_prospecto && a.estado);
        var requestCrosscore = Crosscore.GenerarRequestCross(frontalB64, posteriorB64, selfieB64, token, SecurityUtil.GetIP(), headers);
        var resultCrosscore = Crosscore.ConsultaCrosscore(requestCrosscore);
        if (resultCrosscore.Success)
        {
            prospecto.respuesta_crosscore = resultCrosscore.Decision;
            if (resultCrosscore.Status)
            { //La verificacion de la decicion se realiza en el api con respecto a las zonas parametrizadas
                var Servicios = new ServiciosController();
                var user = new UserDTO()
                {
                    UserName = "ProcesoOriginacionCrosscore",
                    Nombre = "ProcesoOriginacionCroscore"
                };
                response = Originacion.AceptacionHabeasData(token, "SI", context, user, null);
            }
            else
            {
                ActualizarEstadoProspecto(prospecto, Constant.MotivosProspecto.Crosscore, (int)EstadosProspecto.Negado);
                var prospecto2 = Connection.ContextC10.tbl_prospectos.FirstOrDefault(p => p.id_prospecto == prospecto.id_prospecto);
                prospecto2.otp_habeasdata = otpVigente.otp;
                prospecto2.fecha_habeasdata = DateTime.Now;
                prospecto2.ip_habeasdata = SecurityUtil.GetIP();
                prospecto2.token = "";
                var habeasDTO = new HabeasDataDTO()
                {
                    nombreCompleto = prospecto2.nombre + " " + prospecto2.primer_apellido,
                    numeroDocumento = prospecto2.numero_identificacion,
                    codigoOtp = prospecto2.otp_habeasdata,
                    fechaHora = DateTime.Now.ToString(),
                    fechaHoraAceptacion = prospecto2.fecha_habeasdata.ToString(),
                    ipAceptacion = prospecto2.ip_habeasdata
                };
                //metodo de guardar PDF_habeasdata en directorio guardar nombre y ruta en tabla
                response = Reporte.GenerarPdfHabeasData(context, habeasDTO);
                var rutaBbHabeasData = Connection.ContextC10.tbl_rutas_documentos.Where(r => r.numero_identificacion == prospecto.numero_identificacion && r.fk_id_tipodocumento
                                        == (int)TiposDocumentos.Habeas_Data).OrderByDescending(r => r.fecha_creacion).First();
                if (rutaBbHabeasData != null && response.Status)
                {
                    Notification.EnviarCorreoRechazado(prospecto.correo_electronico, rutaBbHabeasData.ruta);
                }

                response.Completed = true;
                response.IsError = false;
                response.Status = true;
                var mensaje = ParameterManager.GetParamsms(Constant.MensajesKey.RespuestaAlCorreo);
                response.Message = mensaje.Value;
                response.MessageCode = mensaje.Codigo;
            }
        }
        else
        {
            prospecto.respuesta_crosscore = resultCrosscore.Decision != null? resultCrosscore.Decision : "" ;
            ActualizarEstadoProspecto(prospecto, Constant.MotivosProspecto.FalloCrosscore, (int)EstadosProspecto.En_Proceso);
            response.Completed = false;
            response.IsError = true;
            response.Status = false;
            var mensaje = ParameterManager.GetParamsms(Constant.MensajesKey.RespuestaDatacreditoDiferente13);
            response.Message = mensaje.Value;
            response.MessageCode = mensaje.Codigo;
        }
        Connection.ContextC10.SaveChanges();

    }
    catch (Exception ex)
    {
        var mensaje = ParameterManager.GetParamsms(Constant.MensajesKey.RespuestaCroscoreDiferenteR0201);
        response.Message = mensaje.Value + " ex " + ex.Message;
        response.MessageCode = mensaje.Codigo;
        response.Completed = false;
        response.IsError = true;
        response.Status = false;
    }
    return response;
}
```

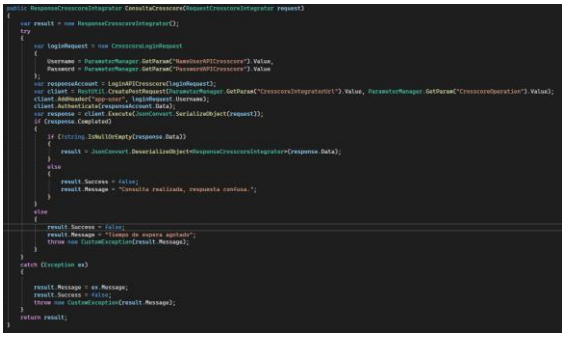
```
public class RequestCrosscoreIntegrator
{
    1 referencia
    public RequestCrosscoreIntegrator( Payload payloadInp, string token, string cedulaProspecto, long idProspecto)
    {
        payload = payloadInp;
        this.token = token;
        this.cedulaProspecto = cedulaProspecto;
        this.idProspecto = idProspecto;
    }

    1 referencia
    public Payload payload { get; set; }
    1 referencia
    public string token { get; set; }
    1 referencia
    public string cedulaProspecto { get; set; }
    1 referencia
    public long idProspecto { get; set; }

}
```

Inicialmente se recupera el prospecto con el token enviado y se establece su estado en “En Proceso” luego se invoca al método “GenerarRequestCross” de CrosscoreService.cs que internamente recuperar las variables Hdim, Jsc y las fotos del prospecto para construir el Request que es de tipo “RequestCrosscoreIntegrator”, una vez se ha obtenido el requestCrosscore con la información del prospecto se invoca al método “ConsultaCrosscore” que retorna un objeto de tipo “ResponseCrosscoreIntegrator”, este objeto tiene las propiedades Status, Success, Decision, Message, responseHeader, clientResponsePayload.

El metodo de BLL “ConsultaCrosscore” del controlador CrosscoreService.cs se encarga de realizar la petición al api propia de Crosscore.Integrator que está desplegada en http://192.168.0.53:44347/, para el consumo se recupera de la tabla [tbl_parametros] el nombre de usuario key_ “NameUserAPICrosscore”, el password key_ “PasswordAPICrosscore”, con estos dos datos se realiza la petición al api para generar el token de acceso usando el metodo “LoginAPICrosscore” por último se recupera la url de consumo key_ “CrosscoreIntegratorUrl” y la operacion key_ “CrosscoreOperation” para ejecutar la solicitud mediante RestUtil de C#.





```
2 referencias
public OperationResponse LoginAPICrosscore(CrosscoreLoginRequest request)
{
    var result = new OperationResponse();
    try
    {
        var host = ParameterManager.GetParamNow("CrosscoreIntegratorLogin").Value;
        var operation = "";
        var client = RestUtil.CreatePostRequest(host, operation);
        var response = client.Execute(JsonConvert.SerializeObject(request));
        if (response.Completed)
        {
            if (!string.IsNullOrEmpty(response.Data))
            {
                var responseLogin = JsonConvert.DeserializeObject<SecurityResponseCrosscore>(response.Data);
                result.Data = responseLogin.token;
            }
            else
            {
                result.Message = "fallo al descerializar";
                result.Status = false;
            }
        }
        else
        {
            response = client.Execute(JsonConvert.SerializeObject(request));
            if (response.Completed)
            {
                if (!string.IsNullOrEmpty(response.Data))
                {
                    var responseLogin = JsonConvert.DeserializeObject<SecurityResponseCrosscore>(response.Data);
                    result.Data = responseLogin.token;
                }
                else
                {
                    result.Message = "fallo al descerializar";
                    result.Status = false;
                }

            }
            else
            {
                throw new CustomException("Error al realizar la consulta");
            }

        }
    }
    catch (Exception ex)
    {
        result.Message = ex.Message;
        result.IsError = true;
    }
    return result;
}
```

Si el consumo del api de Crosscore se realiza de manera satisfactoria se obtiene un true en la propiedad Success de “ResponseCrosscoreIntegrator” de lo contrario se obtiene false. La propiedad Decision muestra la decisión final de Crosscore en forma de texto: las posibles decisiones son las siguientes: “STOP”, “REFER”, “ERROR”, “ACCEPT”. Cuando la decisión es “ACCEPT” quiere decir que el prospecto es viable y paso todas las validaciones de Crosscore en este caso desde el api se retorna la variable Status en True indicando que se aprobó las validaciones de Crosscore.

El método “Flujo Crosscore” evalúa la respuesta de “ConsultaCrosscore” “ResponseCrosscoreIntegrator”, en caso de que Success sea False se establece el estado del prospecto en “En Proceso” y se añade el motivo “FalloCrosscore”, se establece el

mensaje de retorno desde [tbl_parametros_mensajes] key_ "RespuestaDatacreditoDiferente13". Si el Success es True se continúa validando la propiedad Status, si Status es false se establece el estado del prospecto en Rechazado y se agrega el motivo "Crosscore", también se diligencian los campos de la aceptación del habeas data con el OTP que fue creado al inicio en el registro, se borra el token del prospecto, por último, se genera el PDF del habeasData con "Reporte.GenerarPdfHabeasData" y se envía el correo electrónico con la respuesta de rechazado usando el método "Notification.EnviarCorreoRechazado" que extrae la plantilla html desde [tbl_configuracion_mensajeria], con key_ "EmailMessageDeclined", en el correo se adjunta el pdf del HabeasData. Si Status es true, se sabe que Crosscore aprobó al prospecto por lo que se continua con el llamado al método Originacion.AceptacionHabeasData de BLL OriginacionController.cs, este método recibe el token del prospecto, el String "SI" quemado indicando la aceptación del habeasdata, el contexto del controlador y el objeto opcional HabeasDataDTO.

EL método de BLL AceptacionHabeasData() es el punto de unión de Originacion sin Crosscore y el flujo de Originacion con Crosscore, de aquí en adelante para ambos casos se realiza el mismo proceso por lo que se utiliza el mismo método de BLL para realizar las consultas a inspector y datacredito y luego ejecutar las validaciones(Validación del número de celular, validación del correo electrónico, validación de la fecha de expedición), finalmente crear el cliente y el cupo para la respectiva aceptación.

El método AceptacionHabeasData se describe a detalle en el documento técnico de Originacion Expuesto Internet, a manera de resumen de lo que se encarga es de crear el PDF de Habeas data con el método *Reporte.GenerarPdfHabeasData*, realizar la consulta a Inspector y a Datacredito con ayuda del método *ConsultaInspektorCentrales()* y crear el cliente y el cupo con *CrearoActivarCliente* en caso de que el proceso y las validaciones se hallan realizado de manera correcta.

Desde el punto de conexión en adelante se usa BLL.AceptacionHabeasData para las consultas de Inspector y Datacredito, luego en el proyecto Creditos.Web.Originacion en caso de que el cupo sea aprobado, se requiere la aceptación del cupo en el link que contiene el token del cupo, cabe resaltar que el proceso de la aceptación del cupo es único y funciona tanto para el flujo de Originacion y Originacion con Crosscore, la aceptación se da en el proyecto Creditos.Web.Originacion con el token del cupo, el link tiene la siguente forma https://www.credi10.com.co/originacion/aceptacioncupo/validartokenaceptacion?token=TOKENDELPROSPECTO

1.3.5 Log

Durante el proceso de originacion con Crosscore se registra en la tabla *[tbl_logs]* las diferentes acciones que realiza el prospecto para la obtención de su cupo, incluyendo el registro del paso a paso durante las capturas de las fotos. La columna acción almacena la descripción del punto en el proceso de originacion en el que se encuentra el prospecto y el detalle almacena la información relacionada al resultado de la acción, las acciones que corresponden a la originacion se muestran a continuación en la siguiente tabla

accion
Validar Token Captura Fotos
Captura Foto Frontal
Captura Foto Posterior
Captura Selfie
ValidarEntradaEndpointGuardarYConsultar
ResultadoGuardarFotosYConsulta
Buscar Prospecto (Originacion)
Crear Prospecto (Originacion Crosscore)
Validacion TOKEN Prospecto (Originacion)
Validacion OTP Prospecto (Originacion)
Aceptacion HabeasData Prospecto (Originacion)
Validacion TOKEN Aceptacion Cupo (Originacion)
Aceptacion del Cupo (Originacion)

El seguimiento a las anteriores acciones puede ser individual por cada prospecto mediante las siguientes consultas:

- Buscar Prospecto (Originacion)
  SELECT * FROM tbl_logs WHERE accion = 'Buscar Prospecto (Originacion)' and detalle like '%<cedulaProspecto>%';

- Crear Prospecto (Originacion)
  SELECT * FROM tbl_logs WHERE accion = 'Crear Prospecto (Originacion)' and detalle like '%<cedulaProspecto>%';

- Validacion TOKEN Prospecto (Originacion)
  SELECT * FROM tbl_logs WHERE accion = 'Validacion TOKEN Prospecto (Originacion)' and detalle like '%<token de tbl_prospectos>%';

- Validacion OTP Prospecto (Originacion)
  SELECT * FROM tbl_logs WHERE accion = 'Validacion OTP Prospecto (Originacion)' and detalle like '%<correo de tbl_prospectos>%';

- Aceptacion HabeasData Prospecto (Originacion)
  SELECT * FROM tbl_logs WHERE accion = 'Aceptacion HabeasData Prospecto (Originacion)' and detalle like '%<token de tbl_prospectos>%';

- Validacion TOKEN Aceptacion Cupo (Originacion)
  SELECT * FROM tbl_logs WHERE accion = 'Validacion TOKEN Aceptacion | Cupo (Originacion)' and detalle like '%<token de tbl_cupos>%';

- Aceptacion del Cupo (Originacion)
  SELECT * FROM tbl_logs WHERE accion = 'Aceptacion del Cupo (Originacion)' and detalle like '%<token de tbl_cupos>%';

- Validación de entrada a la vista de captura de fotos
  SELECT * FROM tbl_logs WHERE accion = 'Validar Token Captura Fotos' and detalle like '%<cedulaProspecto>%';

- Validación captura foto frontal del documento
  SELECT * FROM tbl_logs WHERE accion = 'Captura Foto Frontal' and detalle like '%<cedulaProspecto>%';

- Validación captura foto posterior del documento
  SELECT * FROM tbl_logs WHERE accion = 'Captura Foto Posterior' and detalle like '%<cedulaProspecto>%';

- Validación captura de la selfie
  SELECT * FROM tbl_logs WHERE accion = 'Captura Selfie' and detalle like '%<cedulaProspecto>%';

- Validación entrada endpoint GuardarFotosYConsulta
  SELECT * FROM tbl_logs WHERE accion = 'ValidarEntradaEndpointGuardarYConsultar' and detalle like '%<cedulaProspecto>%';

- Validación finalización del endpoint GuardarFotosYConsulta
  SELECT * FROM tbl_logs WHERE accion = 'ResultadoGuardarFotosYConsulta' and detalle like '%<cedulaProspecto>%';

1.3.6 Lista de Scripts y la descripción de lo que contiene cada uno

El script de creación de la base de datos: Script Creacion de la Base de Datos